WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

三方库Markdown介绍



主讲人：方维

OHOS Cangjie Markdown

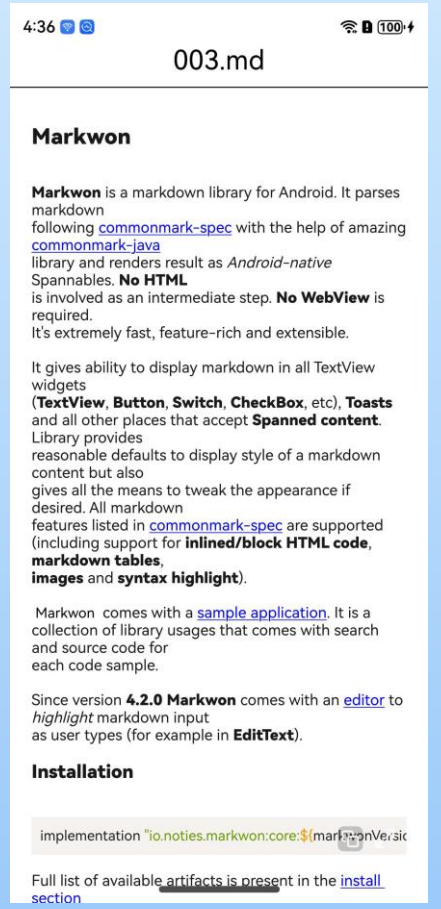
目录

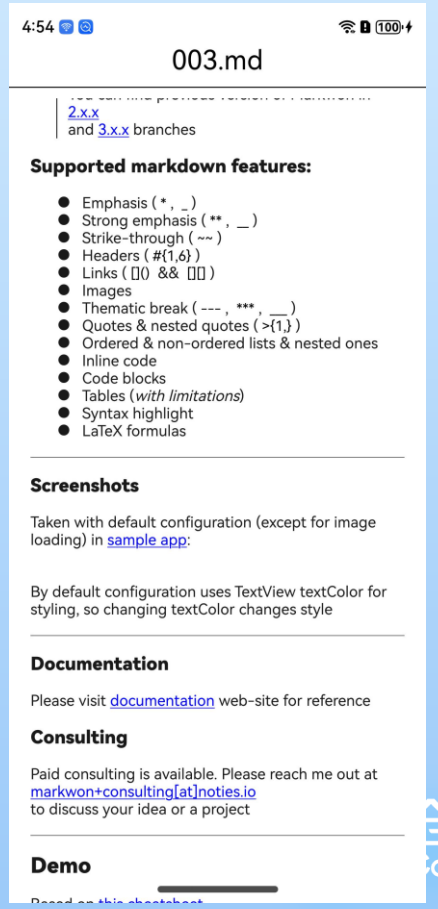
- markdown介绍
- 功能展示
- 集成方法

Markdwn 是一种轻量级的标记语言，可用于纯文本文档中添加格式化元素。Markdown4cj库使用仓颉语言来实现基于commonmark规范的markdown文件解析。将markdown文档转为易于操作的AST(抽象语法树)。利用仓颉UI组件将解析后的markdown转化为高度可定制的富文本，从而实现美观且性能优良的展示效果。

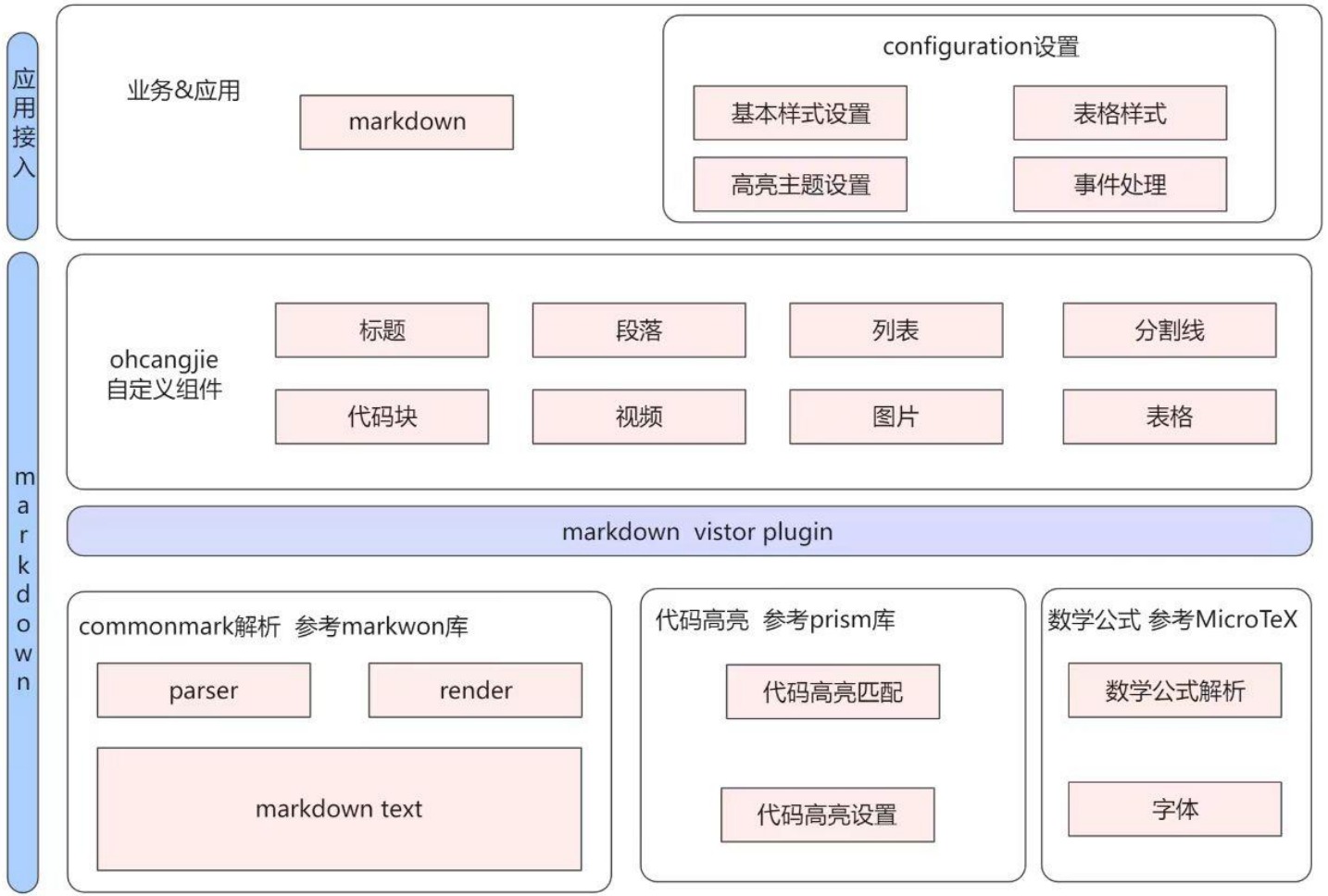
优点

- **使用高效便捷**
- **扩展性强，用户可以自定义Markdown显示样式**
- **符合Markdown规范**



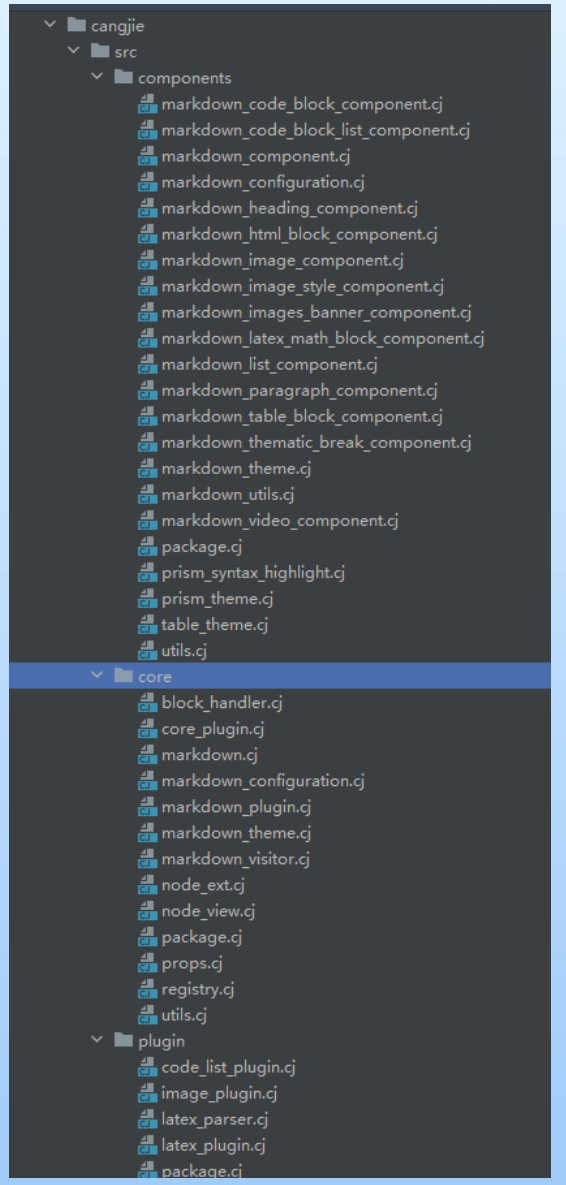


LeetCode


应用接入
业务&应用
markdown
configuration设置
基本样式设置
表格样式
高亮主题设置
事件处理
markdown
ohcangjie
自定义组件
标题
段落
列表
分割线
代码块
视频
图片
表格
markdown vistor plugin
commonmark解析 参考markwon库
parser
render
markdown text
代码高亮 参考prism库
代码高亮匹配
代码高亮设置
数学公式 参考MicroTeX
数学公式解析
字体

Markdown代码解析

```
├─markdown
    └─src
            └─main
              ├─cangjie
              |         └─src
              |             ├─components 自定义UI组件
              |      ├─core markdown 数据解析处理
        |     └─plugin markdown 插件化
        └─resources  资源文件
```



仓颉 Cangjie

示例

基本使用

```
@Entry
@Component
class EmptyPage {
    func render() {
        Scroll() {
            Column {
                MarkdownComponent(mdStr: mdStr)
            }
        }
        // 设置滚动方法
        .scrollable(ScrollDirection.Vertical)
    }

    let mdStr: String =""
}
```

设置样式

```
@Entry
@Component
class TestPage {
    func build() {
        MarkdownComponent(mdStr: mdstr, markdownConfiguration: setConfig())
    }

    func setConfig(): MarkdownConfiguration {
        let markdownConfigurationBuilder: MarkdownConfigurationBuilder = MarkdownConfiguration.emptyBuilder()
        // 设置markdown样式
        markdownConfigurationBuilder.setMarkdownTheme(setMarkdownTheme())
        return markdownConfigurationBuilder.build()
    }

    func setMarkdownTheme(): MarkdownTheme {
        let markdownTheme: MarkdownTheme = MarkdownTheme.emptyBuilder()
            // 设置软换行是否换行
            .setIsLineBreak(true)
            // 设置一级二级标题下划线高度
            .setHeadingBreakHeight(0.0)
            // 设置链接文本颜色
            .setLinkColor(Color.GREEN)
            .build()
        return markdownTheme
    }
}
```

仓颉 Cangjie

标题展示

一级标题

二级标题

三级标题

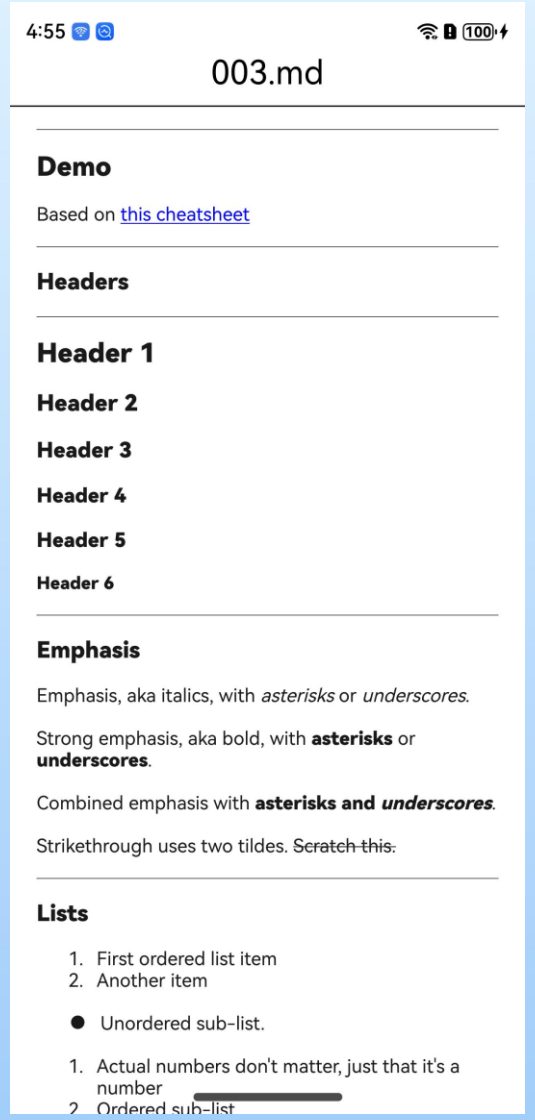
四级标题

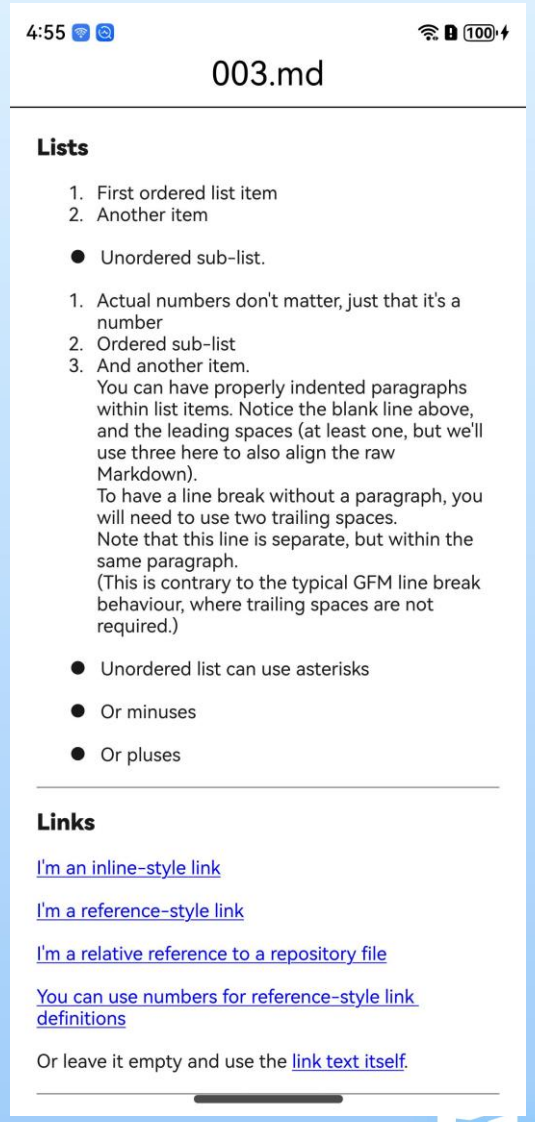
五级标题

六级标题

一级标题
===

二级标题





仓颉 Cangjie

段落展示

普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本。

第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本。

普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本，普通段落文本。

第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，第二段普通段落文本，

```
 * 支持段落语法展示
 */
@Entry
@Component
class ARParagraph2Page {
    func render() {
        Scroll() {
            Column {
                MarkdownComponent(mdStr: mdStr, markdownConfiguration: setConfig())
            }
        }
        // 设置滚动方法
        .scrollable(ScrollDirection.Vertical)
    }

    func setConfig(): MarkdownConfiguration {
        let markdownConfigurationBuilder: MarkdownConfigurationBuilder = MarkdownConfiguration.emptyBuilder()
        markdownConfigurationBuilder.setImageCallback(imageCallback)
        markdownConfigurationBuilder.setLinkCallback(linkCallback)
        markdownConfigurationBuilder.setMarkdownTheme(setMarkdownTheme())
        return markdownConfigurationBuilder.build()
    }

    func setMarkdownTheme(): MarkdownTheme {
        let markdownTheme: MarkdownTheme = MarkdownTheme.emptyBuilder()
                                            // 视频默认占位图
                                            .setVideoImage(@r(app.media.video_image))
                                            // 视频播放按钮默认图
                                            .setPlayCircleFillIcon(@r(app.media.play_circle_fill))
                                            // 代码复制按钮默认图
                                            .setCodeCopyIcon(@r(app.media.icon_copy))
                                            // 代码全屏按钮默认图
                                            .setCodeFullScreenIcon(@r(app.media.full_screen))
                                            // 图片banner默认占位图
                                            .setBannerImage(@r(app.media.video_image))
                                            // 图片默认占位图
                                            .setImageResource(@r(app.media.video_image))
                                            .build()
        return markdownTheme
```

仓颉 Cangjie

列表展示

- 无序列表1
- 无序列表2
- 无序列表3

* 无序列表1
* 无序列表2
* 无序列表3

+ 无序列表1
+ 无序列表2
+ 无序列表3

1. 有序列表1
2. 有序列表2
3. 有序列表3

```
/**
 * 支持无序列表语法
 */
@Entry
@Component
class ARUnorderedList2Page {
    func render() {
        Scroll() {
            Column {
                MarkdownComponent(mdStr: mdStr)
            }
        }
        // 设置滚动方法
        .scrollable(ScrollDirection.Vertical)
    }

    let mdStr: String = """
### 无序列表1

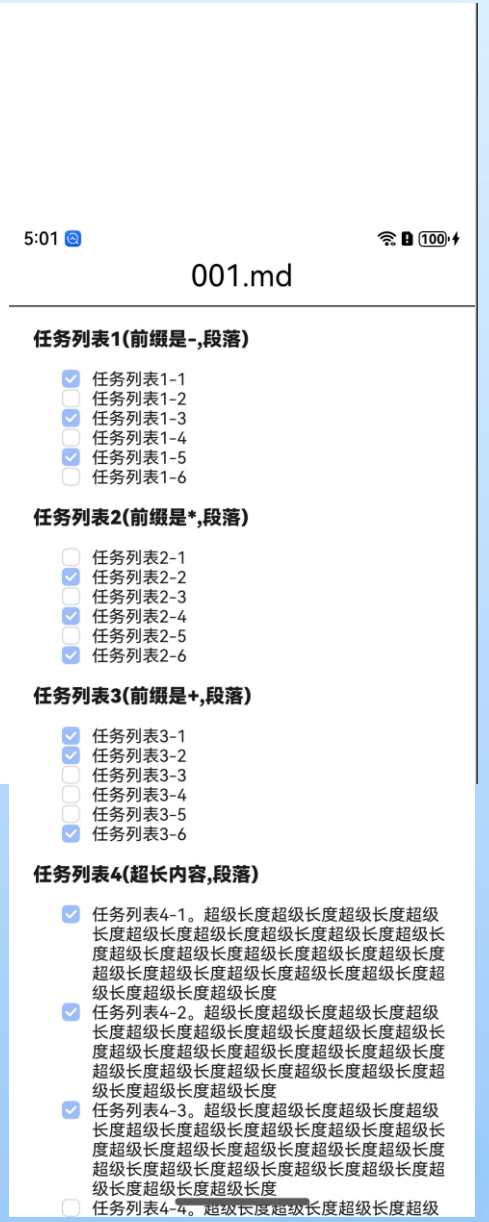
- 无序列表1-1
    - 无序列表1-1-1
    - 无序列表1-1-2
    - 无序列表1-1-3
    - 无序列表1-1-4
    - 无序列表1-1-5
    - 无序列表1-1-6
```

- 无序列表1
- 无序列表2
- 无序列表3

- 无序列表1
- 无序列表2
- 无序列表3

- 无序列表1
- 无序列表2
- 无序列表3

1. 有序列表1
2. 有序列表2
3. 有序列表3


5:01
001.md
任务列表1(前缀是-,段落)
任务列表1-1
任务列表1-2
任务列表1-3
任务列表1-4
任务列表1-5
任务列表1-6
任务列表2(前缀是*,段落)
任务列表2-1
任务列表2-2
任务列表2-3
任务列表2-4
任务列表2-5
任务列表2-6
任务列表3(前缀是+,段落)
任务列表3-1
任务列表3-2
任务列表3-3
任务列表3-4
任务列表3-5
任务列表3-6
任务列表4(超长内容,段落)
任务列表4-1。超级长度超级长度超级长度

TOC目录展示

```md
[TOC]

# 标题1

## 标题2

### 标题3
```

```
/**
 * 支持Toc目录列表语法
 */
@Entry
@Component
class ARTocDirectoryList1Page {
    let scroller = Scroller()
    func render() {
        Scroll(scroller) {
            Column {
                MarkdownComponent(mdStr: mdStr, markdownConfiguration: setConfig())
            }
        }
        // 设置滚动方法
        .scrollable(ScrollDirection.Vertical)
    }

    func setConfig(): MarkdownConfiguration {
        let markdownConfigurationBuilder: MarkdownConfigurationBuilder = MarkdownConfiguration.emptyBuilder()
        markdownConfigurationBuilder.setTocIndexCallback(tocnnn)
        return markdownConfigurationBuilder.build()
    }

    func tocnnn(y:?Float64){
        if(let Some(v) <- y){
            AppLog.debug("------------y:${y}")
            scroller.scrollTo(xOffset: 0.vp, yOffset: px2vp(v.px).getOrThrow())
        }
    }

    let mdStr: String = """
```



难点：
TOC目录中点击跳转到内容节点

- 事件回调
- onAreachange
- 计算控件高度

仓颉 Cangjie

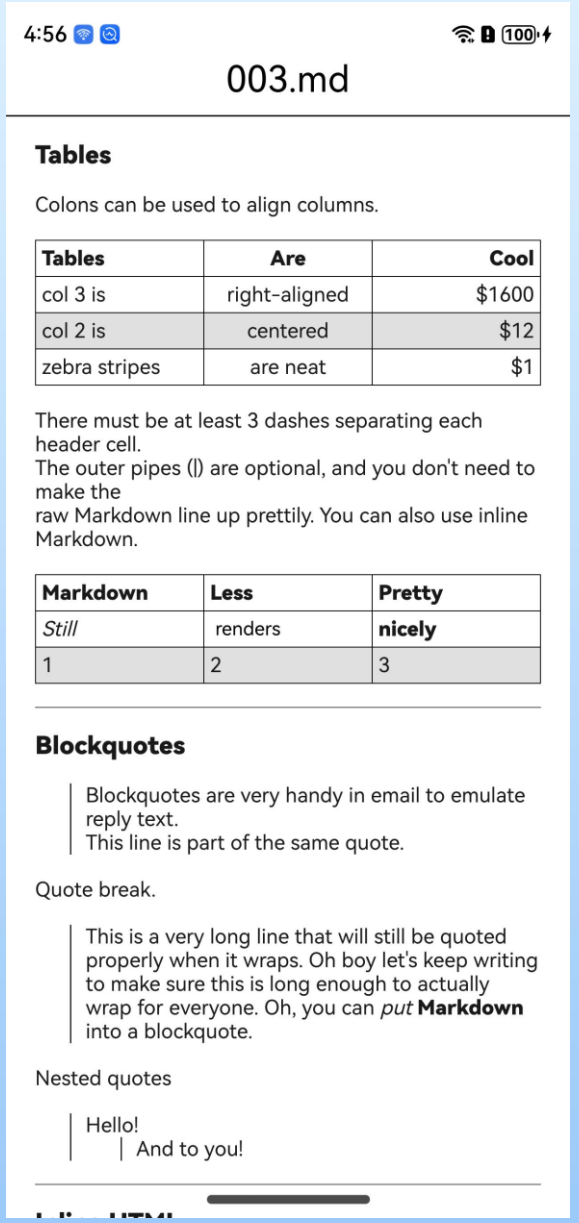
表格展示

序号	标题	难易度
1	两数之和	简单
15	三数之和	中等
262	行程和用户	困难

```
/**
 * 支持表格语法显示
 */
@Entry
@Component
class ARTable1Page {
    func render() {
        Scroll() {
            Column {
                MarkdownComponent(mdStr: mdStr)
            }
        }
        // 设置滚动方法
        .scrollable(ScrollDirection.Vertical)
    }

    let mdStr: String = """
### 表格1

| 题号 | 标题 | 难易度 |
| :--- | ---:| :---: |
| 1 | 两数之和 | 简单 |
| 15 | 三数之和 | 中等 |
| 262 | 行程和用户 | 困难 |
```



仓颉 Cangjie

代码展示

组合代码markdown文本内容

017.md

````
```java [solution1-Java]
public class Main {
    public static void main(String[] args) {
        System.out.print("你好仓颉");
    }
}
```
```python [solution1-Python]
print "你好仓颉"
```
````

组合代码显示截图

Java Python

```
print "你好仓颉"
```

Java Python

```
public class Main {
    public static void main(String[] args) {
        System.out.print("你好仓颉");
    }
}
```

代码

```
@Component
class ARFenceCode1Page {
    func build() {
        Scroll() {
            Column {
                MarkdownComponent(mdStr: mdStr, markdownConfiguration: setConfig())
            }
        }
        // 设置滚动方法
        .scrollable(ScrollDirection.Vertical)
    }

    func setConfig(): MarkdownConfiguration {
        let markdownConfigurationBuilder: MarkdownConfigurationBuilder = MarkdownConfiguration.emptyBuilder()
        markdownConfigurationBuilder.setCodeCopyCallback(codeCopyCallback)
        markdownConfigurationBuilder.setCodeFullScreenCallback(codeFullScreenCallback)
        markdownConfigurationBuilder.setMarkdownTheme(setMarkdownTheme())
        return markdownConfigurationBuilder.build()
    }

    func setMarkdownTheme(): MarkdownTheme {
        let markdownTheme: MarkdownTheme = MarkdownTheme.emptyBuilder()
                                            // 代码复制按钮默认图
                                            .setCodeCopyIcon(@r(app.media.icon_copy))
                                            // 代码全屏按钮默认图
                                            .setCodeFullScreenIcon(@r(app.media.full_screen))
                                            .build()
        return markdownTheme
    }

    func codeCopyCallback(url: String) {
        PromptAction.showToast(message: url)
        AppLog.info("-----------------------codeCopyCallback:${url}")
    }

    func codeFullScreenCallback(url: String, infos: ?String) {
        PromptAction.showToast(message: "${url}${infos}")
        AppLog.info("-----------------------codeFullScreenCallback:${url}")
        AppLog.info("-----------------------codeFullScreenCallback:${infos}")
    }
```

仓颉 Cangjie

Markdown依赖库

CommonMark

01

markdown解析

根据CommonMark规范解析和呈现markdown文本

Prism

02

语法高亮

提供人与语言标记化策略目前支持c、c++、c#、css、dart、go、groovy、java、js、kotlin、latex、python、swift、yaml、cangjie等27种语法高亮显示

Formula

03

数学公式

提供数学公式解析，生成bitmap

. . . . . .

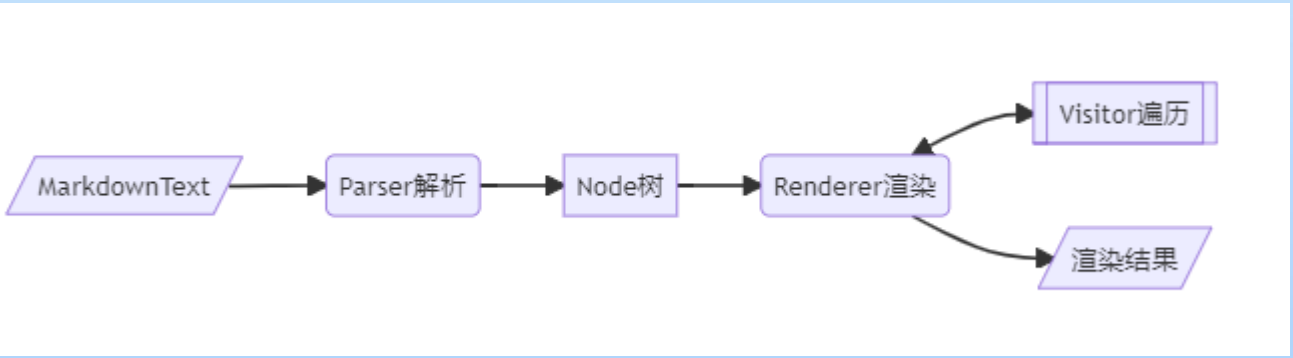
仓颉 Cangjie

commonmark

用于根据CommonMark规范（以及一些扩展）解析和呈现Markdown文本。

特性

- 解析markdown文本
- Node树状结构
- 遍历/渲染Node树




```
main(): Int64 {
    let parser: Parser = Parser.builder().customBlockParserFactory(DashBlockParserFactory()).build()

    let document: Node = parser.parse("hey\n\n---\n")

    println(document.getFirstChild().getOrThrow().toString())
    println((document.getFirstChild().getOrThrow().getFirstChild().getOrThrow() as Text).getOrThrow().getLiteral())
    println(document.getLastChild().getOrThrow().toString())

    return 0
}

class DashBlockParserFactory <: AbstractBlockParserFactory {
    public override func tryStart(state: ParserState, matchedBlockParser: MatchedBlockParser): ?BlockStart {
        if (state.getLine() == ("---")) {
            return BlockStart.of4Cj(DashBlockParser())
        }
        return BlockStart.none()
    }
}

class DashBlock <: CustomBlock {
    public func getNodeType(): NodeType {
        "DashBlock"
    }
}

class DashBlockParser <: AbstractBlockParser {
    private var dash: DashBlock = DashBlock()

    public override func getBlock(): Block {
        return dash
    }

    public override func tryContinue(parserState: ParserState): ?BlockContinue {
        return BlockContinue.none()
    }
}
```

Prism(高亮语法)

特性

- 支持标记不同类型的关键词
- 支持不同语言的解析器/分发器
- 支持预定义语法解析器




```
@Test
public class TestCReadme {

    @TestCase
    public func test01(): Unit {
        let fileArr: Collection<File> = TestUtils.testFiles("c")
        var prism: Prism = Prism(GrammarLocatorGrammarUtils())
        var c: Case
        for (file in fileArr) {
            c = TestUtils.readCase(file)
            match (prism.grammar("c")) {
                case Some(v) => TestUtils.assertCase(c, prism.tokenize(c.input, v))
                case None => ()
            }

        }
    }

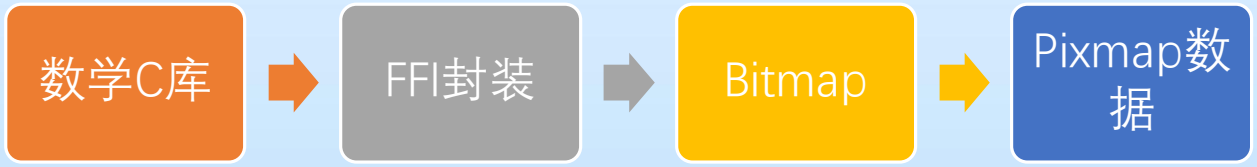
}
```

Latex(数学公式)

用于 LaTeX 编写的数学公式显示。

特性

- 提供生成解析数学公式接口
- 提供生成bitmap接口
- 提供生成图片资源接口




```
import formula.*
import std.fs.*

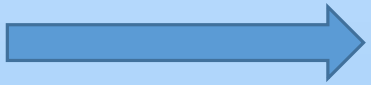
main(): Int64 {

    var latex = LaTeX("res")
    var str = ###"
\sideset{^\backprime}{'}\sum_{x=1}^{\infty} x\sideset{a_1^2}{}\sum_{x=1}^\infty x_0
\\
\sideset{_\text{left bottom}'''}{_{\text{right bottom}}'''}\sum_{\text{quite long text}}^\infty x
\\
\sideset{}{'}
\sum_{n<k,\;\text{$n$ odd}} nE_n
\\
\sideset{}{'}
\sum^{n<k,\;\text{$n$ odd}} nE_n
\\
M_x''' M'''_x M^{'''}_x M_x{'''} M^{\prime\backprime}
"###
    var r = latex.parse(str, 2000, 40.0, 10.0, 0xFF000000)
    var w = r.getWidth()
    var h = r.getHeight()
    var g2 = Graphic2D(r, COLOR_FORMAT_RGB_565)
    r.draw(g2, 0xFFFFFFFF)

    var arr = r.toBitmap(g2)

    var file: File = File("test.bmp", OpenOption.CreateOrTruncate(false))
    file.write(arr)
    file.close()

    return 0
}
```



$$\sum_{x=1}^{\infty}{}' x \, a_1^2 \sum_{x=1}^{\infty} x_0$$

$$\sum_{\substack{\text{left bottom}\\ \text{quite long text}}}^{\infty}{}''' x$$

$$\sum_{n<k,\, n \text{ odd}}{}' nE_n$$

$$\sum_{n<k,\, n \text{ odd}}{}' nE_n$$

$$M_x''' M_x''' M_x''' M_x''' M^{\prime\backprime}$$

接入应用

通过module引入

1、下载项目，将markdown模块拷贝到应用项目下
2、修改自身应用 entry 下的 oh-package.json5 文件，在 dependencies 字段添加 "markdown": "file:../markdown"

```
{
  "name": "entry",
  "version": "1.0.0",
    "description"   :    "Please describe the basic information."   ,
    "main"   :    ""   ,
    "author"   :    ""   ,
    "license"   :    ""   ,
   "dependencies"   :    {
       "markdown"   :    "file:../markdown"
    }
  }
```

3、修改自身应用 entry/src/main/cangjie 下的 cjpm.toml 文件，在 [dependencies] 字段下添加 markdown = {path = "../../../../markdown/src/main/cangjie", version = "1.0.0"}

```
[dependencies]
    markdown = {path = "../../../../markdown/src/main/cangjie", version = "1.0.0"}
```

4、在项目中使用 from markdown import components.* 引用markdown项目

```
internal import markdown.components.*
```

仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，
获取最新官方资讯!

